THREE MODEL MID-SEM PAPERS (Unit-I + Unit-II ONLY)

> **These are practice papers made by Claude from the syllabus and assignment emphasis. They are *not* leaked papers and there is no guarantee any question will appear.** Marks (30 each) and time (about 90 minutes) are assumptions — ask your faculty for the real pattern and scale accordingly. **Attempt each paper on paper, timed, before opening the answer key.**

PAPER 1 — NORMAL DIFFICULTY

Course: CIE-407T Distributed Systems & Cloud Computing · **Syllabus:** Unit-I & II · **Max marks:** 30 · **Time:** 90 min

Q1. Answer any FIVE (2 marks each) — Short-answer / conceptual [10] - (a) Define a distributed system. Give two of its characteristics. - (b) What is marshalling? What is the role of a stub? - (c) State the three rules of the happened-before relation. - (d) What is a critical section? State the safety requirement of distributed mutual exclusion. - (e) Differentiate omission failure and arbitrary failure. - (f) What is the role of the RMI registry?

Q2. Comparison [5] Compare **message passing and shared memory** as IPC mechanisms (working, speed, synchronisation, scalability, applications).

Q3. Numerical [5] Three processes start with Lamport clocks = 0. P1: a (internal), b (send m1 → P2), c (internal), d (receive m3 from P3). P2: e (receive m1), f (internal), g (send m2 → P3). P3: h (internal), i (internal), j (receive m2), k (send m3 → P1). (i) Find the Lamport timestamp of every event. (ii) Are events b and h concurrent? Justify.

Q4. Diagram-based [5] Draw and explain the architecture of Java RMI. Show remote interface, stub, registry, server and client with the steps of execution.

Q5. Application / Case study [5] A college's five lab servers S1–S5 use S5 (highest id) as coordinator. S5 crashes and S2 notices. Using the **Bully algorithm**, show the messages exchanged and the new coordinator. State the best and worst-case message count for N processes.

ANSWER KEY — PAPER 1

Q1. (a) A system whose components on networked computers communicate and coordinate **only by message passing**. Characteristics: concurrency; no global clock; independent failures. (b) **Marshalling** = converting arguments/results to a flat byte format for transmission. **Stub** = client-side proxy that marshals the request, sends it, and unmarshals the reply, hiding remote communication. (c) HB1: events in the same process are ordered by

occurrence. HB2: send(m) → receive(m). HB3: transitivity (a→b, b→c ⇒ a→c). (d) **Critical section** = code accessing a shared resource. Safety: at most **one** process in the CS at a time. (e) Omission: process crashes or message lost (fails to act). Arbitrary/Byzantine: behaves incorrectly – wrong or contradictory values, corrupts messages; hardest to handle. (f) The RMI registry is a naming service (port 1099) mapping names to stubs; the server `rebind`s, the client `lookup`s.

Q2. Definition of each (message passing = explicit send/receive; shared memory = common region read/written with locks) + table: | Point | Message passing | Shared memory | |---|---|---| | Location | same or different machines | same machine (or DSM) | | Speed | slower (copy, network) | faster | | Synchronisation | implicit in blocking send/receive | explicit locks/semaphores | | Scalability | high | low | | Isolation/security | better | poor | | Applications | client-server, chat, MPI | multithreading, OS shared segments | Conclusion: DS uses message passing since nodes have no common physical memory. (Diagram of both: 1 mark.)

Q3. | Event | Rule | L | |---|---|---| | a | 0+1 | 1 | | b (send m1, ts=2) | 1+1 | 2 | | e (recv m1) | max(0,2)+1 | 3 | | f | 3+1 | 4 | | g (send m2, ts=5) | 4+1 | 5 | | h | 0+1 | 1 | | i | 1+1 | 2 | | j (recv m2 ts=5) | max(2,5)+1 | 6 | | k (send m3 ts=7) | 6+1 | 7 | | c | 2+1 | 3 | | d (recv m3 ts=7) | max(3,7)+1 | 8 | (ii) b → e → f → g → j → k → d is a causal chain, but there's **no message path from b to h or h to b** (h is before j only). So **b ‖ h** (concurrent) even though L(h)=1 < L(b)=2: Lamport clocks cannot show concurrency.

Q4. Marking: diagram (2), components (1.5), steps (1.5). Diagram: Server (remote object) ←→ RMI Registry (1099) ←→ Client (stub). Steps: (1) define remote interface extends Remote; (2) implementation extends UnicastRemoteObject; (3) server creates object and `rebind("Name",obj)`; (4) client `lookup("Name")` → stub; (5) client calls method on stub; (6) stub marshals & sends; (7) server unmarshals, invokes real object; (8) result returned, unmarshalled. (See `01 §12`.)

Q5. S2 → ELECTION to S3, S4, S5 (3). S3, S4 → OK (2). S5 crashed (silent). S3 → ELECTION to S4, S5 (2); S4 → OK to S3 (1). S4 → ELECTION to S5 (1) → timeout, no reply. S4 announces **COORDINATOR(S4)** to S1, S2, S3 (3). **New coordinator = S4.** Total = 12 messages. Best case: second-highest process detects → **N−2** coordinator messages; worst case: lowest-id process detects → **O(N²)**.

PAPER 2 — MODERATE DIFFICULTY

Max marks: 30 · **Time:** 90 min

Q1. Answer any FOUR (2 marks each) [8] - (a) What is middleware? Why is it needed? - (b) Distinguish synchronous and asynchronous distributed systems. - (c) Write the three invocation semantics of RPC/RMI and name the one used by Java RMI. - (d) Define clock skew and clock drift. - (e) What is a consistent cut? - (f) List the properties of reliable multicast.

Q2. Diagram-based / Conceptual [5] Explain **architectural models** of distributed systems. Draw diagrams for (i) software layers with middleware, (ii) client–server with multiple servers, (iii) proxy/cache.

Q3. Numerical [5] (a) *Cristian's algorithm:* client sends request at T0 = 14:00:00.000, receives reply at T1 = 14:00:01.000; server time in reply = 14:00:20.500. Compute the client's new time and the error bound if the minimum one-way delay is 0.1 s. (b) *Berkeley algorithm:* master clock 5:00, slave A 4:52, slave B 5:14. Compute each machine's adjustment.

Q4. Comparison [4] Compare **logical time and physical time** (any 6 points) and mention one algorithm for each.

Q5. Case study [8] Three bank servers P1, P2, P3 initially hold ₹600, ₹400, ₹200 (total ₹1200). P1 sends ₹100 to P2 (m1); P2 sends ₹50 to P3 (m2). P3 initiates a snapshot after m2 was sent but before it arrives. Using the **Chandy–Lamport algorithm** (FIFO channels) show the markers, the recorded local states and channel states, and verify that the total is ₹1200. State the assumptions of the algorithm.

ANSWER KEY — PAPER 2

Q1. (a) Software layer between OS and applications masking heterogeneity (hardware, OS, network, languages) and giving a uniform programming model (RMI, RPC, CORBA). (b) Synchronous: known bounds on execution time, message delay, drift → timeouts detect failures. Asynchronous: no bounds (Internet) → slow vs crashed indistinguishable. (c) Maybe, at-least-once, at-most-once; Java RMI = **at-most-once**. (d) Skew = difference between two clocks' readings at the same instant. Drift = rate at which a clock deviates from a perfect clock. (e) A cut in which for every receive event inside the cut its send event is also inside (no orphan message). (f) Integrity, validity, agreement.

Q2. (Refer `01 §5.2`.) Answer skeleton: definition (architectural = structure/placement of components); (i) layers diagram: Applications → Middleware → OS → Hardware (platform = OS + HW); (ii) multiple servers: clients → server 1/2/3 cooperating (partition/replicate); (iii) proxy: client → proxy/cache → server (reduces latency & load); add P2P, mobile code, mobile agents, thin clients in one line each; conclusion. Marks: 3 diagrams (2.5) + explanation (2.5).

Q3. (a) RTT = 1.000 s; new time = 20.500 + 0.500 = **14:00:21.000**. Accuracy = ±(RTT/2 − min) = ±(0.5 − 0.1) = **±0.4 s**. (b) Offsets: master 0, A −8, B +14 (minutes). Average = (0 − 8 + 14)/3 = **+2 min** → target time 5:02. Adjustments: **master +2, A +10, B −12** minutes.

Q4. Table (`02 §8`): meaning; based on; purpose; sync needed; affected by drift; example; relation to real time; detects concurrency. Algorithms: physical → Cristian/Berkeley/NTP; logical → Lamport clock (or vector clock).

Q5. 1. P1 sends m1 (₹100): P1 = 500. P2 sends m2 (₹50): P2 = 350. 2. P3 (initiator): records **S3 = 200**, sends MARKERs on C31, C32; starts recording C13, C23. 3. m2 arrives at P3 (before P2's marker on C23) → recorded into channel C23. 4. P1 receives first MARKER (on C31): records **S1 = 500**, C31 = ∅, sends MARKERs on C12 (after m1) and C13; records C21. 5. P2 receives P3's MARKER (C32) [before m1 arrives]: first marker → records **S2 = 350**, C32 = ∅, sends MARKERs on C21, C23; starts recording C12. 6. m1 arrives at P2 while recording C12 → then P1's MARKER on C12 → **C12 = {₹100}**. 7. P3 receives P2's MARKER on C23 → **C23 = {₹50}** (m2 arrived while recording). P3 receives P1's marker on C13 → C13 = ∅. P1 receives P2's marker on C21 → C21 = ∅. **Recorded state:** S1 = 500, S2 = 350, S3 = 200, C12 = {100}, C23 = {50}. Sum = 500+350+200+100+50 = **₹1200** ✓. Assumptions: FIFO, reliable,

unidirectional channels; no process/channel failure during snapshot; strongly connected network; any process may initiate. (Diagram + message flow: marks included.)

PAPER 3 — TOUGH / CONCEPTUAL

Max marks: 30 · **Time:** 90 min

Q1. Answer any FOUR – reasoning questions (2 marks each) [8] - (a) Lamport clocks satisfy a→b ⇒ L(a)<L(b). Why can we not conclude a→b from L(a)<L(b)? Give a small example. - (b) Why must channels be FIFO in the Chandy–Lamport algorithm? What may go wrong otherwise? - (c) Why does the Bully algorithm assume a synchronous system, whereas the Ring algorithm does not? - (d) Why can Byzantine agreement with one traitor not be reached among three generals? - (e) Why must a clock never be set backwards during synchronisation? What is done instead? - (f) Why is total-ordered multicast harder to implement than FIFO-ordered multicast?

Q2. Analysis [4] "An asynchronous system with crash failures cannot guarantee consensus." Explain this statement, and describe two practical techniques used to work around it.

Q3. Numerical / concurrency detection [5] Use **vector clocks** for 3 processes (initially (0,0,0); increment own component before every event): P1: a (internal), b (send m1 → P2), c (receive m3 from P3). P2: d (receive m1), e (send m2 → P3). P3: f (internal), g (receive m2), h (send m3 → P1). (i) Compute the vector timestamp of every event. (ii) Show that a → h. (iii) Show that f and b are concurrent. (iv) Compute Lamport timestamps for the same events and identify a pair where Lamport clocks mislead.

Q4. Application / Comparison [5] Six servers share a log file that is updated a few times per hour. Compare **central-server, token-ring, Ricart–Agrawala and Lamport's** mutual-exclusion algorithms for N = 6 on message count, synchronisation delay and failure behaviour. Recommend one with justification.

Q5. Case study [8] A replicated database has 7 processes that must agree on a commit/abort decision. (a) If up to 2 processes may **crash** (synchronous system), how many rounds are needed and how does the flooding algorithm reach consensus? (b) If up to 2 processes may be **Byzantine**, is consensus possible with 7 processes? With 6? Why? (c) State the three conditions of consensus and the effect of unbounded message delay.

ANSWER KEY — PAPER 3

Q1. (a) The clock condition works only one way. Example: P1 executes local event x (L=1); P2 executes event y (L=2) after a previous local event; no message between them → L(x)<L(y) but x ∥ y. Only vector clocks capture both directions. (b) The marker separates messages sent **before** the sender recorded its state from those **after**. With non-FIFO channels a message sent after the marker could overtake it and be counted as "before", or the reverse, producing an inconsistent state (lost or double-counted money). (c) Bully depends on **timeouts** to decide that a higher-ID process is dead (no OK ⇒ declare self coordinator) – reliable only if delays are bounded (synchronous). Ring never waits for absence of a reply; it passes an ID around and a process wins only when its own ID returns, so it does not need timing bounds (only a repaired

ring). (d) With 3 generals and 1 traitor a loyal lieutenant cannot tell whether the commander or the other lieutenant is lying (symmetric information) — Byzantine agreement needs N ≥ 3f+1 = 4. (e) Setting the clock backwards would make time repeat and break ordering (e.g., file timestamps, `make`, logs). Instead the clock is **slowed down** (lower tick rate) until it catches up. (f) FIFO needs only per-sender sequence numbers; total order requires all processes to **agree on one global order** — needs a sequencer (SPOF/bottleneck) or an agreement protocol (ISIS) ⇒ effectively consensus.

Q2. FLP (Fischer–Lynch–Paterson 1985): in a purely asynchronous system, no deterministic algorithm can guarantee termination if even one process may crash, because a slow process cannot be distinguished from a crashed one; waiting forever violates termination, deciding early risks disagreement. Workarounds: (i) **failure detectors / timeouts** (partial synchrony, e.g., Paxos/Raft leader election with timeouts); (ii) **randomisation** (probabilistic termination); also (iii) assume synchrony bounds.

Q3. (i) | Event | Vector | |---|---| | a | (1,0,0) | | b (send m1) | (2,0,0) | | d (recv m1) | max((0,0,0), (2,0,0)) = (2,0,0) → own+1 = **(2,1,0)** | | e (send m2) | **(2,2,0)** | | f | **(0,0,1)** | | g (recv m2) | max((0,0,1),(2,2,0)) = (2,2,1) → +1 = **(2,2,2)** | | h (send m3) | **(2,2,3)** | | c (recv m3) | max((2,0,0),(2,2,3)) = (2,2,3) → +1 = **(3,2,3)** | (ii) V(a) = (1,0,0) < V(h) = (2,2,3) (every component ≤, one strictly <) ⇒ **a → h**. (iii) V(f) = (0,0,1), V(b) = (2,0,0): f ≰ b (third component 1 > 0) and b ≰ f (first 2 > 0) ⇒ **f ‖ b**. (iv) Lamport: a=1, b=2; d=max(0,2)+1=3, e=4; f=1, g=max(1,4)+1=5, h=6; c=max(2,6)+1=7. Note L(f)=1 < L(b)=2, but f ‖ b ⇒ Lamport misleads (suggests f "before" b).

Q4. (N=6) | Algorithm | Msgs/entry | Sync delay | Failure behaviour | |---|---|---|---| | Central server | 3 | 2 msg times | Server crash stops all (SPOF); needs election | | Token ring | 1–5 (+ constant circulation) | 1–5 | Token loss / node crash breaks ring | | Ricart–Agrawala | 2(N−1) = 10 | 1 | Any crash ⇒ no reply ⇒ block | | Lamport | 3(N−1) = 15 | 1 | Same, plus needs FIFO | **Recommendation:** since updates are rare (few per hour), the **central server** is best (3 messages, simple, fair); a dedicated coordinator (elected via Bully if it fails) gives fault tolerance. Ring wastes bandwidth circulating a token when nobody wants the CS; RA/Lamport are unnecessarily heavy.

Q5. (a) f = 2 crashes ⇒ **f + 1 = 3 rounds**. Each process keeps a set of values; in each round multicasts the values not yet sent, adds received values; after round 3 decides deterministically (e.g., min or majority) of its set. Since among 3 rounds at least one has no crash, all correct processes have identical sets ⇒ agreement. (b) Byzantine needs **N ≥ 3f + 1**: f = 2 ⇒ N ≥ 7. **7 processes: possible** (with f+1 = 3 rounds of message exchange, oral messages OM(2)). **6 processes: impossible** (unsigned messages) — traitors can create indistinguishable views. (c) Termination (every correct process eventually decides), Agreement (same value), Validity/Integrity (decided value was proposed; if all propose v, decision = v). Unbounded delay ⇒ system asynchronous ⇒ FLP: cannot distinguish slow from crashed; must use timeouts/randomisation, accepting possible non-termination in rare runs or weaker guarantees.

Self-evaluation table

Paper	Marks /30	Weak topics to revise
1		

Paper	Marks /30	Weak topics to revise
2		
3		